

Bản trình chiếu: Thuật toán di truyền

Qian Ziqiang

2005

Nguồn gốc: Phân tích và nghiên cứu ứng dụng của thuật toán di truyền: bản trình chiếu

IOI China candidate team papers / 2005 / Qian Ziqiang / Qian Ziqiang.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết về thuật toán di truyền. Lớp văn bản của slide còn đọc được các mốc: *Genetic Algorithm*, John Holland, IOI 2005, các thuật toán Prim và Kruskal, một ví dụ có số 1200, cùng nhiều hình và biểu đồ không trích được ổn định từ tệp PowerPoint cũ. Bản ghi chú này dựa trên bài viết đã dịch để phục hồi nội dung slide ở mức ý tưởng: giới thiệu thuật toán di truyền, nêu quy trình, rồi dùng bài cây khung nhỏ nhất đa mục tiêu để minh họa.

2 Tư tưởng

Thuật toán di truyền mô phỏng tiến hóa: mỗi cá thể biểu diễn một nghiệm, hàm thích nghi đánh giá chất lượng, rồi quần thể được cập nhật qua chọn lọc, lai ghép và đột biến. Cách tiếp cận này thường dùng cho bài tối ưu khó, nơi lời giải chính xác quá đắt hoặc không cần thiết.

Điểm khác biệt của GA so với tìm kiếm cục bộ đơn lẻ là nó làm việc trên cả một quần thể. Mỗi cá thể là một điểm trong không gian nghiệm; qua nhiều thế hệ, những cá thể có độ thích nghi cao có xác suất đóng góp nhiều hơn vào thế hệ sau. Nhờ vậy thuật toán vừa khai thác các nghiệm tốt đã tìm thấy, vừa giữ một mức ngẫu nhiên để thoát khỏi cực trị cục bộ.

Quy trình cơ bản gồm năm bước:

1. chọn cách mã hóa nghiệm thành nhiễm sắc thể;
2. sinh quần thể ban đầu;
3. tính độ thích nghi của từng cá thể;
4. tạo thế hệ mới bằng giữ lại, lai ghép và đột biến;
5. lặp cho tới khi hết số thế hệ hoặc hội tụ đủ chậm.

Slide nhấn mạnh rằng GA không phải phép màu thay cho mô hình hóa. Nếu mã hóa không bảo toàn cấu trúc của nghiệm, hoặc hàm thích nghi không phản ánh đúng mục tiêu, quần thể có thể tiến hóa rất nhanh nhưng đi về hướng sai.

3 Bài cây khung nhỏ nhất đa mục tiêu

Việc nhắc tới Prim và Kruskal cho thấy slide có ví dụ liên quan tới cây khung trên đồ thị. Với cây khung nhỏ nhất một mục tiêu, Prim và Kruskal chọn cạnh theo trọng số và cho lời giải tối ưu hiệu quả. Nhưng trong bài toán thực tế, mỗi cạnh có thể có nhiều trọng số: chi phí, độ ổn định, độ khó thi công, độ trễ, tác động môi trường, v.v. Khi ấy ta không còn một tổng trọng số duy nhất để tối thiểu hóa.

Bài toán trong bài viết là cây khung nhỏ nhất đa mục tiêu. Một nghiệm là một cây khung, còn giá trị của nghiệm là vector

$$F(x) = (f_1(x), f_2(x), \dots, f_m(x)),$$

trong đó $f_i(x)$ là tổng chi phí của cây trên mục tiêu thứ i . Ta quan tâm các nghiệm không bị trội: không có nghiệm khác tốt hơn hoặc bằng trên mọi mục tiêu và tốt hơn hẳn trên ít nhất một mục tiêu. Lớp bài toán này là NP-khó, nên vét cạn hoặc cố ép về một mục tiêu bằng trọng số tuyến tính đều có hạn chế.

4 Mã hóa bằng mã Prüfer

Để dùng GA, trước hết phải mã hóa cây khung thành nhiễm sắc thể. Slide đi theo mã Prüfer: với đồ thị đầy đủ n đỉnh, mỗi cây tương ứng với một chuỗi độ dài $n - 2$, mỗi phần tử là một số trong $1, \dots, n$. Biểu diễn này có hai lợi ích thực dụng. Thứ nhất, sinh một nhiễm sắc thể ngẫu nhiên rất dễ: chỉ cần sinh $n - 2$ số. Thứ hai, giải mã một chuỗi thành cây luôn cho một cây hợp lệ, nên các thao tác di truyền không cần sửa chữa quá nhiều nghiệm không hợp lệ.

Bản slide gốc có hình minh họa cây và chuỗi Prüfer tương ứng; ở đây không lặp lại hình. Ý chính là: khi mã hóa, liên tục xóa lá có nhãn nhỏ nhất và ghi lại đỉnh kề nó; khi giải mã, dùng phần tử đầu của chuỗi cùng đỉnh nhỏ nhất chưa xuất hiện trong chuỗi để phục hồi cạnh. Phần này là nền để hiểu vì sao lai ghép và đột biến trên chuỗi vẫn tạo ra cây khung sau khi giải mã.

5 Hàm thích nghi và toán tử

Với nhiều mục tiêu, không nên cộng thô các chi phí vì thang đo của từng mục tiêu có thể rất khác nhau. Bài viết dùng hàm đánh giá dạng

$$g(x) = \sum_{i=1}^m \left(\frac{\min[i]}{f_i(x)} \cdot 100 \right)^2,$$

trong đó $\min[i]$ là giá trị tốt nhất đã thấy trên mục tiêu thứ i . Cách chuẩn hóa này giúp cá thể tốt ở một mục tiêu không bị che mất chỉ vì mục tiêu khác có miền giá trị lớn hơn.

Ba nhóm toán tử chính là:

- **giữ lại:** đưa cá thể tốt sang thế hệ sau để hỗ trợ hội tụ;
- **lai ghép:** hoán đổi các đoạn ngắn cùng vị trí giữa hai chuỗi Prüfer, rồi giữ cá thể con tốt hơn;
- **đột biến:** thay ngẫu nhiên một vị trí trong chuỗi, thường với xác suất nhỏ để tránh phá hỏng cấu trúc tốt đã có.

Các tỷ lệ trong bài viết là giữ lại 54%, lai ghép 45%, đột biến 1%. Slide chỉ cần người đọc nhớ vai trò cân bằng: giữ lại giúp hội tụ, lai ghép khai thác nghiệm hiện có, đột biến duy trì khả năng tìm kiếm toàn cục.

6 Thử nghiệm và cách đọc kết quả

Các biểu đồ trong slide dùng để so sánh GA với tìm kiếm nguyên thủy trên dữ liệu nhỏ, vừa và một số dữ liệu có tương quan giữa các mục tiêu. Với dữ liệu rất nhỏ, tìm kiếm nguyên thủy có thể xác nhận nghiệm tối ưu. Khi kích thước tăng, không gian cây khung tăng rất nhanh; bài viết ghi những trường hợp tìm kiếm mất hàng chục phút, nhiều giờ, thậm chí ước lượng hàng trăm năm, trong khi GA vẫn cho nghiệm gần tối ưu trong thời gian ngắn.

Mốc 1200 đọc được trong slide nhiều khả năng thuộc phần minh họa dữ liệu hoặc tham số thử nghiệm. Điều quan trọng hơn là cách đánh giá: thuật toán di truyền không hứa tìm toàn bộ tập Pareto tối ưu một cách chứng minh được trong mọi lần chạy. Nó cho một nhóm nghiệm tốt, thường rất gần biên tối ưu, và có thể chạy lại nhiều lần để tăng xác suất tìm được nghiệm mong muốn.

7 Ghi chú về mã nguồn

Tài liệu gốc có phụ lục chương trình Pascal khá dài. Theo tinh thần bản ghi chú slide, không cần chép lại khuôn mã ấy. Các điểm cần giữ khi cài đặt là: mã hóa và giải mã Prüfer, tính vector chi phí của cây, cập nhật tập nghiệm không bị trội, chọn cá thể theo độ thích nghi, rồi áp dụng ba toán tử với tỷ lệ đã đặt.

8 Kết luận

Thuật toán di truyền mạnh ở khả năng tìm nghiệm tốt trong không gian lớn, nhưng cần thiết kế mã hóa và hàm thích nghi cẩn thận. Bài trình bày nhấn mạnh đây là công cụ heuristic, không phải thay thế cho chứng minh tối ưu khi bài đòi hỏi đáp án chính xác. Với bài cây khung đa mục tiêu, thông điệp của slide rất rõ: Prim và Kruskal giải đẹp bài một mục tiêu, còn khi mục tiêu trở thành một vector và không gian nghiệm quá lớn, tư tưởng tiến hóa là một hướng thực dụng để tìm nhanh các nghiệm tốt.